

Lab 4: PySpark

26AI5703 – Data Engineering & Big Data Systems Lab

Aim: Process the supplied datasets using Spark RDDs. The code and results are presented below in task order.

1a. Spark installation and test

Spark 4.2.0 and Java 21 were installed. The Spark shell was started with:

```
spark-shell --master 'local[2]'
```

Shell command: `sc.parallelize(Seq(1, 2, 3)).sum()`. Output: 6.0. The equivalent Python check and setup are:

```
import findspark
findspark.init()

from operator import add
from pyspark import SparkConf, SparkContext

conf = (SparkConf().setMaster("local[2]").setAppName("Lab4")
        .set("spark.hadoop.fs.defaultFS", "file:///")) # Read local files, not HDFS.
sc = SparkContext.getOrCreate(conf)
sc.setLogLevel("ERROR")
print("Spark:", sc.version, "| Test sum:", sc.parallelize([1, 2, 3]).sum())
```

Output:

Spark: 4.2.0 | Test sum: 6

1b. Word count

`flatMap` splits lines into words. Each word becomes `(word, 1)`, and `reduceByKey(add)` adds the counts of repeated words. The sample file contains `hello spark`, `hello hadoop`, and `spark spark` on three lines. The earlier Hadoop input was not supplied.

```
word_counts = (sc.textFile("data/wordcount.txt").flatMap(str.split)
               .map(lambda word: (word, 1)).reduceByKey(add)
               .sortByKey().collect())
print(word_counts)
```

Output:

Word	Count
hadoop	1
hello	2
spark	3

1c. Rating distribution for every movie

MovieLens 100K has 100,000 ratings for 1,682 movies. Count each `(movieID, rating)` pair, then collect the five counts per movie. Missing ratings have count zero.

```
ratings = sc.textFile("data/ml-100k/u.data").map(str.split)
rating_counts = (ratings.map(lambda r: ((int(r[1]), int(r[2])), 1))
                 .reduceByKey(add))
movie_counts = (rating_counts.map(lambda x: (x[0][0], (x[0][1], x[1])))
                .groupByKey().mapValues(dict).sortByKey().collect())
movie_distribution = [(movie, [counts.get(rating, 0) for rating in range(1, 6)])
                      for movie, counts in movie_counts]
print(movie_distribution[:10])
```

Output, first five movies (from the ten-row preview):

Movie ID	1 star	2 stars	3 stars	4 stars	5 stars
1	8	27	96	202	119
2	8	17	55	42	9
3	11	20	25	23	11
4	6	24	57	93	29
5	4	11	32	33	6

2. Average number of friends for every age

Input columns: ID, name, age, and number of friends (no header). The dataset contains 500 people across 52 ages. `mapValues` changes only the value of a key-value pair; the age key stays the same. It first changes a friend count to `(count, 1)`. `reduceByKey` adds both parts for each age, and the second `mapValues` calculates:

$$\text{average friends at an age} = \frac{\text{total friends of people of that age}}{\text{number of people of that age}}.$$

```
friends = sc.textFile("data/friends_test.csv").map(lambda line: line.split(","))
average_friends = (friends.map(lambda r: (int(r[2]), int(r[3])))
    .mapValues(lambda count: (count, 1))
    .reduceByKey(lambda a, b: (a[0] + b[0], a[1] + b[1]))
    .mapValues(lambda total: total[0] / total[1])
    .sortByKey().collect())
for age, average in average_friends:
    print(age, round(average, 2))
```

Output: all ages, rounded to two decimal places.

Age	Average	Age	Average	Age	Average
18	343.38	36	246.60	54	278.08
19	213.27	37	249.33	55	295.54
20	165.00	38	193.53	56	306.67
21	350.88	39	169.29	57	258.83
22	206.43	40	250.82	58	116.55
23	246.30	41	268.56	59	220.00
24	233.80	42	303.50	60	202.71
25	197.45	43	230.57	61	256.22
26	242.06	44	282.17	62	220.77
27	228.12	45	309.54	63	384.00
28	209.10	46	223.69	64	281.33
29	215.92	47	233.22	65	298.20
30	235.82	48	281.40	66	276.44
31	267.25	49	184.67	67	214.62
32	207.91	50	254.60	68	269.60
33	325.33	51	302.14	69	235.20
34	245.50	52	340.64		
35	211.62	53	222.86		

For example, people aged 18 have an average of 343.38 friends. This is an average across those people, not the friend count of one person.

3 and 4. Minimum and maximum temperatures

The CSV columns are `itemID`, `stationID`, `desc`, `temp`. Skip the header and convert `temp` to an integer. `filter` keeps only the requested measurement type: `TMIN` for minimum temperatures, or `TMAX` for maximum temperatures. Precipitation records are excluded.

```
lines = sc.textFile("data/temp.csv")
header = lines.first()
temperatures = (lines.filter(lambda line: line != header)
                 .map(lambda line: line.split(","))
                 .map(lambda r: (r[0], r[1], r[2], int(r[3]))))

def temperature_extrema(kind, operation):
    selected = temperatures.filter(lambda r: r[2] == kind)
    overall = selected.map(lambda r: r[3]).reduce(operation)
    by_item = selected.map(lambda r: (r[0], r[3])).reduceByKey(operation).sortByKey().collect()
    by_station = selected.map(lambda r: (r[1], r[3])).reduceByKey(operation).sortByKey().collect()
    print("Overall:", overall)
    print("By item:", by_item)
    print("By station:", by_station)
    return overall, by_item, by_station

minimum_temperatures = temperature_extrema("TMIN", min)
maximum_temperatures = temperature_extrema("TMAX", max)
```

`reduce` combines all selected temperatures into one result. `reduceByKey` combines only temperatures with the same item or station key. The shared function uses `min` for task 3 and `max` for task 4.

Output: overall and every item with temperature records.

Group	Minimum (TMIN)	Maximum (TMAX)
Overall	-148	323
EZE00100082	-135	323
ITE00100554	-148	323

Output: first five station keys, out of 365.

Station ID	Minimum (TMIN)	Maximum (TMAX)
18000101	-148	-75
18000102	-130	-44
18000103	-73	-10
18000104	-74	0
18000105	-58	10

Temperatures retain the supplied numeric units. Grouping follows the CSV headers, including the date-like values in `stationID`.

Stop Spark after completing all tasks:

```
sc.stop()
```

Result: All tasks ran successfully. Outputs matched independent Python calculations; all movie rating counts together total 100,000.

Source: GroupLens MovieLens 100K. The accompanying code PDF contains the notebook cells and saved outputs.